Pymoo_and_the_front

- The Pareto front is the set of solutions where you can't improve one objective without worsening another. Every point on it is a best-possible trade-off. In our case, we have $(1 - \text{worst task acc}, \log \text{wiring cost}, \log \text{firing cost})$ as the 3 objectives to be minimized.
- NSGA-III is an evolutionary algorithm: it keeps a population of candidate solution and improves over generation. It does the iteration of breed (make children from current population) → evaluate (each child on the objective) → rank (check if each child is on the front or non-dominated) → keep the best points
- How is the pareto front calculated here:
  - candidate pair of wiring and firing budgets $(\max \text{wiring cost}, \max \text{firing cost})$
  - train RNN based on those costs (using gradient descent: Adam) and measure $(1 - \text{worst task acc}, \log \text{wiring cost}, \log \text{firing cost})$
  - a network only counts if $\text{worst task acc} > 0.6$
  - at the end, all networks ever trained are pooled, not just the last generation. Failing (infeasible) networks are dropped.
- Why not use NSGA-III directly to optimize all 3 objectives?
  - Making $(1 - \text{worst task acc})$ lower means setting ~90k weights correctly. Doing that via NSGA-III means evolving them by trials and error. Each training step would mean evolution of that network each time: million times breed → evaluate →rank loop
  - gradient descent does it better as backprop tells the weight which direction to move in.
  - NSGA-III handles cost tradeoffs to try. In short, NSGA-III picks the costs, gradient descent achieves the best accuracy at those costs, and the front comes from combining the two.
- How wiring and firing costs ceilings are achieved:
  - Wiring (hard bound): after every training step, if $\frac{1}{N}\sum w_{\text{ij}} > C_{\text{wire}}$ every synapse is shrunk by same amount until sum fits under threshold

$$| W_{ij} | \to \max(| W_{ij} | - \tau, 0)$$

  - Firing (soft bound): activity cannot be clipped automatically, so a penalty on firing is added to the loss and firing adjusts automatically. The firing rate settles near b_r

$$L = L_{\text{task}} + \lambda C_{\text{rate}}$$
$$\log \lambda \to \log \lambda + \eta \left(\frac{C_{\text{rate}}}{b_r} - 1 \right)$$